

Diseño de software: proceso y productos

Elaborado por Gabriel Alberto García Mireles

Semestre 2021-2

Asignatura: Análisis y Diseño Orientado a Objetos

Objetivos

- Conocer el proceso del diseño
 - Los productos que resultan de este proceso servirán de base para la construcción del software
 - Se deberán considerar los principios de diseño, ambiente de implementación y estándar de diseño
- Definir la arquitectura del software
 - Se establecen los componentes principales del software y la relación entre ellos
- Diseñar componentes de software
 - Modelado estático y dinámico de cada componente
- Definir el orden de integración de componentes y su prueba

Objetivos

Objetivo	Actividades	Productos
O1. Conocer el proceso de diseño	1. Entender los principios de diseño 2. Determinar el ambiente de operación 3. Establecer el estándar de diseño	1. Especificación del ambiente de implementación 2. Estándar de documentación del diseño
O2. Definir la arquitectura del software	1. Identificar la arquitectura del software 2. Elaborar el diagrama de distribución	1. Arquitectura con diagrama de paquetes 2. Diagrama de distribución
O3. Diseñar componentes software	1. Modelar la vista estática del componente 2. Modelar la vista dinámica del componente 3. Modelar la navegación en la interfaz de usuario	1. Diagrama de clases 2. Diagrama de secuencia 3. Diagrama de estados
O4. Definir el orden de integración de componentes y su prueba	1. Crear plan de pruebas de integración	1. Plan de pruebas de integración

O1. Conocer el proceso de diseño

- El diseño del software es un proceso creativo para decidir **cómo se construirá el software**
- **El diseño se desarrolla proponiendo soluciones y refinando estas soluciones.**
- **El proceso de diseño tiene dos niveles de abstracción**
 - **Diseño arquitectónico**
 - Identifica los componentes principales a partir de la especificación de requisitos
 - Determina las relaciones entre los componentes principales
 - **Diseño detallado**
 - Especifica en detalle cada componente principal considerando la vista estática y la dinámica de sus partes constituyentes

O1. Conocer el proceso de diseño

- Principios de diseño
 - Diseñar para el cambio
 - Los requisitos del software cambian con frecuencia
 - Es necesario anticiparte a los cambios con un diseño flexible que facilite la actualización del software
 - Diseñar para facilitar el uso del software
 - Tener en cuenta a los usuarios del software.
 - Identificar sus conocimientos, habilidades y actitudes
 - Diseñar escenarios apropiados para cada perfil de usuario
 - Diseñar para facilitar la prueba
 - Elaborar componentes que puedan ser probados de manera independientes
 - Diseño para la reutilización
 - Identificar los componentes genéricos que puedan volver a usarse (código, casos de prueba, modelos, diagramas)

Principios del diseño



Programador,
desarrollador de
software,
Especialista en
mantenimiento

Diseñar para el
cambio

Diseñar para
reusar

Facilidad de uso

Facilidad de prueba

Pueden usarse para
evaluar la calidad del
diseño



Tipos de usuarios



Especialista en pruebas

O1. Conocer el proceso de diseño

- Medición del diseño
 - Cohesión (Mejor: alta cohesión)
 - Grado de relación entre los elementos que pertenecen a un componente
 - Criterio para determinar la cohesión
 - Redactar el objetivo del componente. Si el enunciado tiene un solo verbo, el componente tiene un buen grado de cohesión. Si el enunciado tiene más de un verbo, o tiene comas, probablemente tiene una cohesión débil y habría que definir un componente para cada verbo
 - Acoplamiento (Mejor: bajo acoplamiento)
 - Grado de relación entre los componentes
 - Criterio para determinar el acoplamiento
 - Número de relaciones de cada componente
 - Un buen diseño tiene acoplamiento bajo

O1. Conocer el proceso de diseño

- Especificación del ambiente de implementación
 - Se toman en cuenta los requisitos del software
 - Requisitos no funcionales,
 - Contexto de uso del sistema
 - Tecnología disponible en el lugar de instalación del software (o acordada con el cliente)
 - Se debe definir
 - El ambiente de desarrollo y su versión
 - Las herramientas de diagramación para UML
 - Servidor de base de datos y su versión
 - Servidor web y su versión
 - Herramientas para automatizar las pruebas unitarias
 - Otros
 - La selección de tecnologías se debe instalar en las estaciones de trabajo correspondientes (o servidores)

O1. Conocer el proceso de diseño

- Estándar de documentación del diseño
 - Facilita el trabajo en equipo
 - Permite integrar documentos y productos con facilidad
 - El estándar incluye
 - **Denominación** de elementos del diseño
 - Componentes, clases variables, métodos, y otros.
 - La convención se puede basarse en estándares como UML o de programación (e.g. Java)
 - Estándar de diseño de la **interfaz de usuario**
 - Página principal y sus menús, diseño gráfico, colores, nombres de botones, fuentes, entre otros
 - Estándar de **mensajes de error** que se presentarán en el sistema

O2. Arquitectura del software

- Consiste en identificar los componentes principales del software y la relación que existe entre ellos
- Un sistema tiene distintas arquitecturas, según la perspectiva de modelado que esté bajo estudio
- Definición de arquitectura de software
 - La arquitectura de software de un programa o sistema de cómputo es la estructura o **estructuras** del sistemas, las cuales se componen de **elementos** de software, las **propiedades externamente visibles** de esos elementos, y las **relaciones** entre ellos

O2. Arquitectura del software

- Ventajas de diseñar explícitamente y documentar la arquitectura
 - Comunicación con los interesados
 - Descripción de alto nivel del sistema que puede usarse como base de discusión entre los interesados
 - Análisis del sistema
 - Orienta el análisis para abordar requisitos críticos como rendimiento, fiabilidad y mantenibilidad
 - Reutilización
 - La arquitectura es una descripción compacta de cómo se organiza un sistema y cómo interoperan sus componentes
 - La arquitectura del sistema es generalmente la misma para sistemas con requisitos similares: líneas de producto

02. Arquitectura del software

- La arquitectura del software afecta el rendimiento, protección, seguridad, disponibilidad y mantenibilidad, entre otros aspectos
 - Rendimiento
 - Las operaciones críticas deberían realizarse en un pequeño número de subsistemas con poca comunicación como sea posible. Uso de componentes de grano grueso
 - Seguridad informática (security)
 - Se recomienda arquitectura en capas, con los recursos más críticos protegidos en las capas más internas y aplicando una validación de seguridad de alto nivel en dichas capas
 - Seguridad física (safety)
 - Las operaciones relacionadas con la seguridad deberán localizarse en un único subsistema o en un número pequeño de ellos. Reduce los costes y los problemas de validación de seguridad
 - Disponibilidad
 - Arquitectura debería incluir componentes redundantes. Reemplazar y actualizar componentes sin detener el sistema
 - Mantenibilidad
 - Diseño de componentes independientes de grano fino que puedan modificarse con facilidad. Evitar estructuras compartidas. Productores de dato deberían separarse de consumidores de datos.

O2. Arquitectura del software

- Cualidades deseables de la arquitectura del software para un sistema particular
 - Sencillez
 - Fácil de comprender y de implementar
 - Extensión
 - La posibilidad de agregar nuevos componentes
 - Cambio
 - Que los cambios en los requisitos no afecten mucho a la arquitectura
- Estilos arquitectónicos (Sommerville, 2005;p224-229)
 - El modelo de repositorio
 - El modelo cliente servidor
 - **El modelo de capas (máquina abstracta)**

O2. Arquitectura del software

- Vista
 - Es una representación de un conjunto coherente de elementos arquitectónicos, como la que escriben y leen los involucrados en el sistema
- Estructura
 - Es el conjunto de elementos, tal como existen en el software o hardware
- Estructura de módulos
 - Los elementos son módulos, los cuales son unidades de implementación
 - Cada módulo tiene asignadas ciertas áreas de funcionalidad. Contesta las siguientes preguntas
 - Cuál es la responsabilidad funcional principal asignada a cada módulo?
 - Con qué otros módulos tiene permitido el uso?
 - Qué otro software realmente lo usa?

O2. Arquitectura del software

- Modelo en capas
 - Una capa es un conjunto coherente de funcionalidad (Bass, 2003)
 - En una estructura estricta por capas,
 - La capa n sólo podría usar los servicios de la capa $n - 1$
 - Las capas, con frecuencia, se diseñan como abstracciones (máquinas virtuales) que ocultan detalles de implementación de las capas inferiores a las superiores. Promueven la portabilidad
 - La regla de este modelo es que cada capa se comunica solamente con capas adyacentes

O2. Arquitectura del software

- Modelo de capas
 - Presentación
 - Maneja la interacción entre el usuario y el software
 - Puede ser una línea de órdenes, menú basado en texto, interfaz gráfica (*rich client*) o una interfaz de usuario para navegador basada en HTML
 - Responsabilidad es desplegar la información al usuario e interpretar las órdenes del usuario en acciones sobre el dominio y la fuente de datos
 - Dominio
 - Este es el trabajo que la aplicación necesita realizar para el dominio en el que estás trabajando.
 - Cálculos basado en datos de entrada y datos almacenadas validación de datos de entrada, determinar qué acciones realizar según las órdenes recibidas de la interfaz
 - Fuente de datos
 - Se comunica con otros sistemas para realizar las tareas a nombre de la aplicación. Estos pueden ser monitores de transacciones, otras aplicaciones, sistemas de mensajes, entre otros.
 - La base de datos es responsable primaria de la persistencia de datos

O2. Arquitectura del software

- Ventajas (Sommerville,2005)
 - Soporta el desarrollo incremental de sistemas
 - Soporta bien los cambios y es portable
 - Cuando las interfaces cambian, sólo se ve afectada la capa adyacente
 - Fácil implementar el sistema en múltiples plataformas
- Desventajas
 - La estructuración de los sistemas puede resultar difícil
 - Las capas internas pueden proporcionar facilidades básicas que son requeridas por todos los niveles
 - Los servicios requeridos por un usuario puede atravesar todas las capas para satisfacer las necesidades de éste
 - Rendimiento problemático
 - Puede tener múltiples niveles de interpretación de órdenes.

02. Arquitectura del software

- El desarrollo de software de aplicación
 - El sistema de información implica interacción con una base de datos compartida
 - El desarrollo de la WWW llevó a la transformación de sistemas de información especializados a sistemas de propósito general universalmente accesibles
 - Los sistemas se pueden modelar aplicando el modelo en capas
 - **Capa de presentación:** en esta capa se colocan los elementos con los que interactúa directamente el usuario: formularios, informes, etc.
 - **Capa lógica de aplicación:** en esta capa se implementan las reglas del negocio y la lógica de la aplicación
 - **Capa de almacenamiento:** contiene los elementos que guardan la información para asegurar su persistencia, como bases de datos o archivos
 - Se representa con un diagrama de paquetes de UML

02. Arquitectura del software

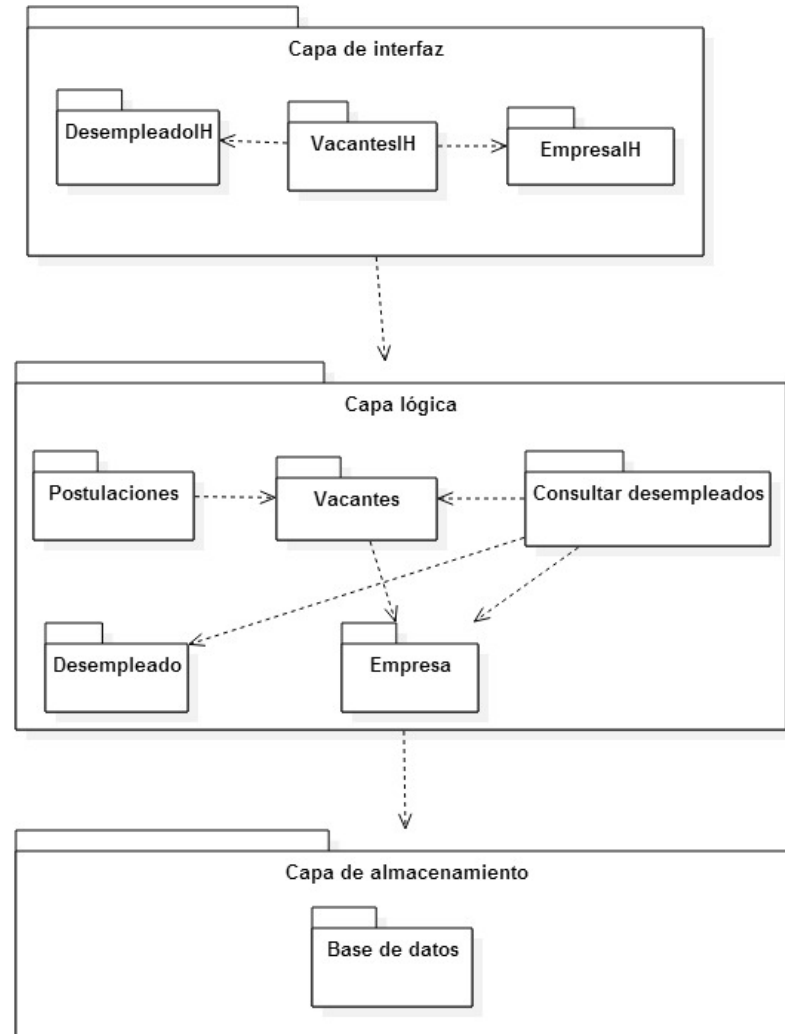
- Diagrama de paquetes (dimensión lógica)
 - Es un conjunto de paquetes y las relaciones que existen entre ellos
- Paquete
 - Es un mecanismo de propósito general para **organizar** los elementos de un modelo en grupos.
 - Permiten comprender más fácilmente un sistema
 - Permiten controlar el acceso a sus contenidos
 - Un paquete se representa en UML como una carpeta con su nombre
 - El diagrama de paquetes muestra la relación entre los paquetes



02. Arquitectura del software

- Contenido del paquete
 - Clases, interfaces, componentes, nodos, colaboraciones, casos de uso, diagramas y otros paquetes
 - Cada elemento pertenece exclusivamente a un único paquete
 - Se puede controlar la visibilidad de los elementos del paquete
- Un paquete forma un espacio de nombres
 - Los elementos contenidos en el paquete deben tener nombres únicos
- Composición jerárquica de paquetes
 - Se refiere al caso en que al menos un paquete contiene otro paquete.
- Mostrar el contenido de un paquete
 - Textualmente o gráficamente

02. Arquitectura del software



O2. Arquitectura del software

- Diagrama de distribución (dimensión física)
 - Un sistema distribuido se ejecuta en más de una computadora
 - Los componentes se encuentran distribuidos en ellas
 - El diagrama de distribución representa los componentes que se instalarán en cada máquina y las conexiones entre ellos
 - Un diagrama de distribución (**deployment**) tiene nodos y conexiones
 - Nodo
 - Elemento físico existente y representa un recurso computacional. Se representa por un cubo
 - Conexión
 - Relación que representa las comunicaciones entre los nodos. Pueden etiquetarse con un protocolo de comunicación

O2. Arquitectura del software

- Nodos y artefactos

- Artefactos

- Son los elemento que participan en la ejecución de un sistema
 - Representan el empaquetamiento físico de los elementos lógicos
 - Son las cosas que se ejecutan en los nodos
 - Es la manifestación de un conjunto de elementos lógicos, como clases y colaboraciones
 - Un conjunto de objetos o artefactos asignados a un nodo como un grupo se denomina unidad de distribución

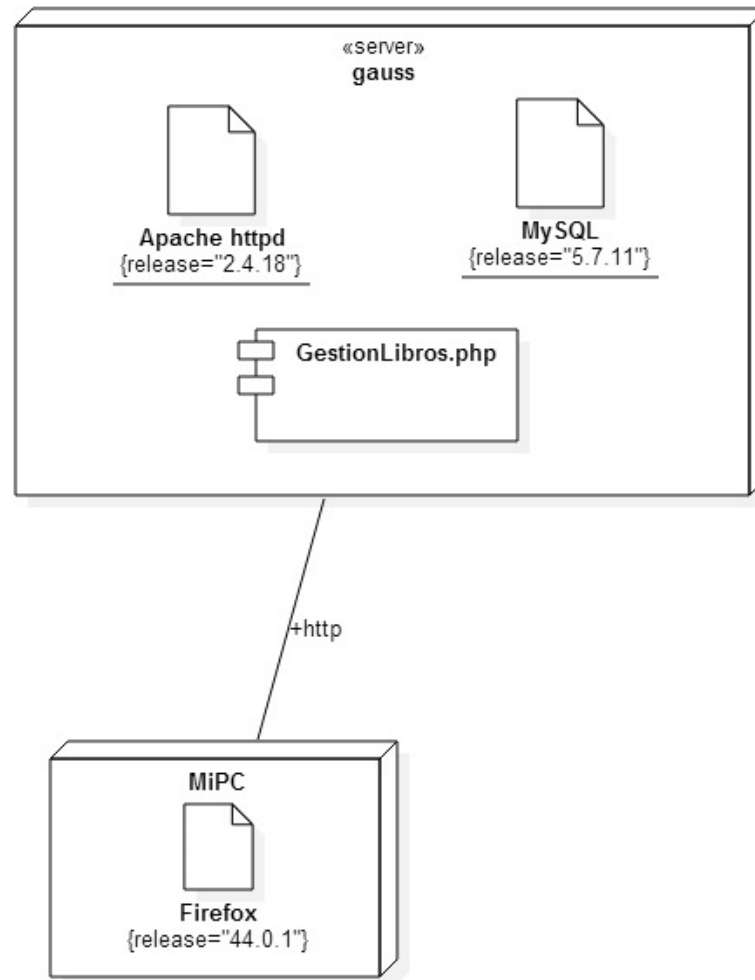
- Nodos

- Son los elementos en donde se ejecutan los artefactos
 - Representan el despliegue físico de los artefactos
 - Permiten la ejecución de los artefactos
 - Es la localización sobre la cual se despliegan los aretfactos

O2. Arquitectura del software

- Conexión
 - El tipo más común de relación entre nodos es la asociación
 - Una asociación representa una conexión física entre nodos
 - Ethernet
 - Línea serie RS-232
 - Un bus compartido
 - Un protocolo de comunicación
- Procesador y dispositivo
 - Un procesador es un nodo con capacidad de procesamiento (puede ejecutar un artefacto)
 - Un dispositivo es un nodo sin capacidad de procesamiento (al menos ninguna modelada a este nivel de abstracción)
 - Representa algo que interactúa con el mundo real

02. Arquitectura del software



02. Arquitectura del software

- Diagrama de despliegue
 - Presenta una vista estática de la configuración en tiempo de ejecución de los nodos procesadores y de los components que corren sobre ellos.
 - Hardware y el software que contienen
 - Se desarrollan cuando la aplicación se distribuye en distintas computadoras, se explora la arquitectura de sistemas embebidos, sistemas distribuidos o sistemas cliente - servidor
- Consideraciones para el diseño de diagramas de despliegue
 - Identificar los elementos computacionales
 - Identificar procesadores y dispositivos
 - Considerar atributos y operaciones que se pueden aplicar en cada nodo
 - Ubicar cada artefacto significativo del sistema en un nodo determinado
 - Un artefacto puede residir en múltiples nodos

O2. Arquitectura del software

- Proceso para definir la arquitectura
 1. Seleccionar el tipo de arquitectura del software según la aplicación
 2. Identificar los paquetes del sistema
 3. Construir el diagrama de paquetes
 4. En el caso de que el sistema sea distribuido, identificar la lógica de la distribución
 5. Construir el diagrama de distribución y asignar los paquetes a los nodos

03. Diseño de componentes

- Componente
 - Es un bloque de construcción modular del software (y sistema)
 - “an entity with discrete structure, such as an assembly or software module, within a system considered at a particular level of analysis” (ISO 25010)
 - “una parte modular, distribuible y reemplazable de un sistema que encapsula la implementación y expone un conjunto de interfaces” (OMG).
 - En el paradigma Orientado a Objetos
 - Un componente contiene un conjunto de clases que colaboran entre sí.
 - Se definen todos los atributos y operaciones relevantes para su implementación
 - Se definen todas las interfaces que permiten que las clases se comuniquen entre componentes
 - La identificación de las clases se realiza analizando la especificación de requisitos y se agregan clases de infraestructura

O3. Diseño de componentes

- Los elementos de los paquetes serán clases
- Se construyen dos vistas de diseño
 - Vista estática
 - Diagramas de clase (representa los elementos estructurales y sus relaciones)
 - Vista dinámica
 - Diagramas de secuencia (representan las interacciones de los objetos)

03. Diseño de componentes

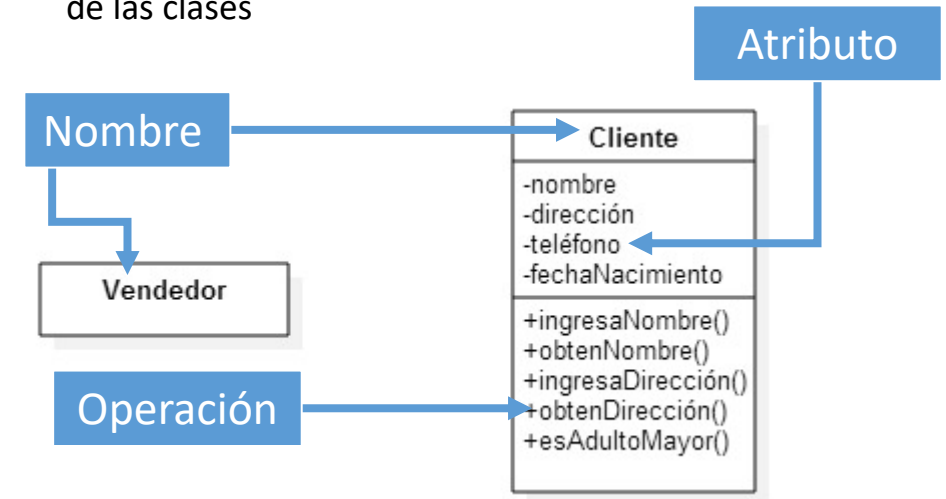
- Los distintos tipos de clases que se modelan en la arquitectura basada en capas son:
 - Interfaz
 - Interfaz con el usuario. Son los elementos con los que el usuario interacciona directamente
 - Ventanas, páginas, informes
 - Corresponden a la capa de presentación y las clases puede llevar el sufijo IH para diferenciarlas de los otros estereotipos
 - Control
 - Reglas del negocio o aplicación. Estas clases implementan las reglas del negocio y la lógica de aplicación
 - Corresponden a la capa de lógica de la aplicación
 - Entidad
 - Base de datos, archivos. Son los elementos que guardan la información para asegurar su persistencia
 - Las clases de este tipo pertenecen a la capa de almacenamiento. Las clases pueden tener el sufijo BD

03. Diseño de componentes

- Diagrama de clases
 - Diagrama que muestra un conjunto de clases, interfaces y colaboraciones y sus relaciones (Booch et al., 2006)
- Clase
 - Descripción de un conjunto de objetos que comparten los mismos atributos, operaciones, relaciones y semánticas (Booch et al., 2006)
 - Se representa gráficamente como un rectángulo

- Elementos de la clase (Booch, 2006)

- Nombre
 - Es una cadena de texto
- Atributo
 - Es una propiedad de una clase identificada con un nombre, que describe un rango de valores que pueden tomar las instancias de la propiedad
- Operación
 - Es la implementación de algo que puede hacer un objeto y que es compartido por todos los objetos de las clases



03. Diseño de componentes

- Relación
 - Conexión semántica entre elementos
- Tipos de relaciones
 - Dependencias 
 - Son relaciones de uso
 - Asociación 
 - Son relaciones estructurales entre instancias
 - Agregación 
 - Especifica una relación todo-parte entre el agregado (el todo) y un componente (la parte)
 - Composición 
 - Forma de agregación con fuerte pertenencia y un tiempo de vida coincidente entre las partes y el todo
 - Generalización 
 - Son relaciones que conectan clases generales con otras más especializadas
 - Realización 
 - Un elemento especifica un contrato que otro clasificador se compromete a llevar a cabo

03. Diseño de componentes

- Dependencia
 - Es una relación de uso que declara que un elemento utiliza la información y los servicios de otro elemento, pero no necesariamente a la inversa
 - Se representa gráficamente con una línea discontinua dirigida hacia el elemento del cual depende
 - Se utilizan cuando se quiere indicar que n elemento utiliza a otro
 - En el diagrama de clases
 - Indican que una clase utiliza las operaciones de otra
 - Utiliza variables o parámetros cuyo tipo viene dado por la otra clase

03. Diseño de componentes

- Generalización
 - Es una relación entre un elemento general (superclase o padre) y un caso más específico de ese elemento (subclase o hijo)
 - También se le conoce como relación “es un tipo de”
 - Ejemplo: Un gato “es un tipo de” felino
 - La generalización significa que el hijo puede sustituir al padre
 - Un hijo hereda las propiedades de sus padres
 - El hijo puede añadir atributos y operaciones
 - Una implementación de una operación en un hijo redefine la implementación de la misma operación en el padre (polimorfismo) Se requiere la misma signature *vs sobrecarga* (diferentes parámetros)
 - Una clase sin padres y uno o más hijos se denomina clase raíz o clase base.
 - Una clase sin hijos se llama clase hoja
 - Una clase con un único padre se dice que utiliza herencia simple
 - Una clase con más de un padre se dice que utiliza herencia múltiple
 - Se representa gráficamente como una línea dirigida continua, con una gran punta de flecha vacía apuntando al padre

03. Diseño de componentes

- Asociación
 - Es una relación estructural que especifica que los objetos de un elemento están conectados con los objetos de otro
 - Es válido que los extremos de una asociación estén conectados a la misma clase
 - Puede existir más de una asociación entre las mismas clases
 - Se representa con una línea continua que conecta la misma o diferentes clases
 - Se utilizan cuando se desea representar relaciones estructurales

03. Diseño de componentes

- Las asociaciones pueden ser descritas por los siguientes elementos
 - Nombre
 - Describe la naturaleza de la relación
 - Se puede utilizar una flecha para dar sentido a la lectura de la asociación
 - Se incluye el nombre cuando los roles no han sido especificados
 - Rol
 - Cuando una clase participa en una asociación, tiene un rol específico
 - Un rol es simplemente la cara que la clase de un extremo de la asociación presenta a la clase del otro extremo
 - Se denomina nombre de extremo
 - Multiplicidad
 - Especificación del rango de cardinalidades (número de elementos) permisible que puede asumir un conjunto
 - Se escribe como una expresión con un valor mínimo y un máximo separados por dos puntos consecutivos: 1..* (1 a muchos) 0..* (cero a muchos o sólo *). Un rango de enteros: (3..5). Un número exacto (2..2).

03. Diseño de componentes

- Asociación
 - Representa una relación estructural entre iguales
 - Ambas clases están en el mismo nivel y tienen al misma importancia
- Agregación
 - Representa una relación “todo/parte” una cosa grande (el todo) consta de elementos más pequeños (las partes)
 - Un objeto del todo tiene objetos de la parte
 - Es un tipo especial de asociación
 - Se añade en la línea de la asociación un rombo vacío en la parte del todo
 - La relación sólo es conceptual, no es semántica
- Composición
 - Es una forma de agregación con una **fuerte relación de pertenencia y vidas coincidentes** de la parte con el todo
 - Las partes con una multiplicidad no fijada pueden crearse después de la parte compuesta, pero una vez creadas viven y mueren con ella
 - Las partes se pueden eliminar explícitamente antes de la eliminación de la parte compuesta
 - La parte compuesta es responsable de disponer de las partes. Debe gestionar la creación y destrucción de éstas

03. Diseño de componentes

- Identificación de clases
 - Se analiza cada caso de uso para imagina qué clases se necesitan según los estereotipos: interfaz, control, persistencia
 - Se inicia identificando las clases control
 - Se identifican los nombres o frases nominales que se usan en el vocabulario del dominio de problema
 - Si el nombre identificado es una cadena o un número es muy probable que sea un atributo
 - Técnica: tarjetas Clase-Responsabilidad-Colaboración (Tarjetas CRC)
 - Se usan para determinar tanto las clases o responsabilidades
 - Para cada responsabilidad se busca la colaboración con otras clases, se pueden generar otras clases

Nombre de la clase:	
Responsabilidad	Colaboración

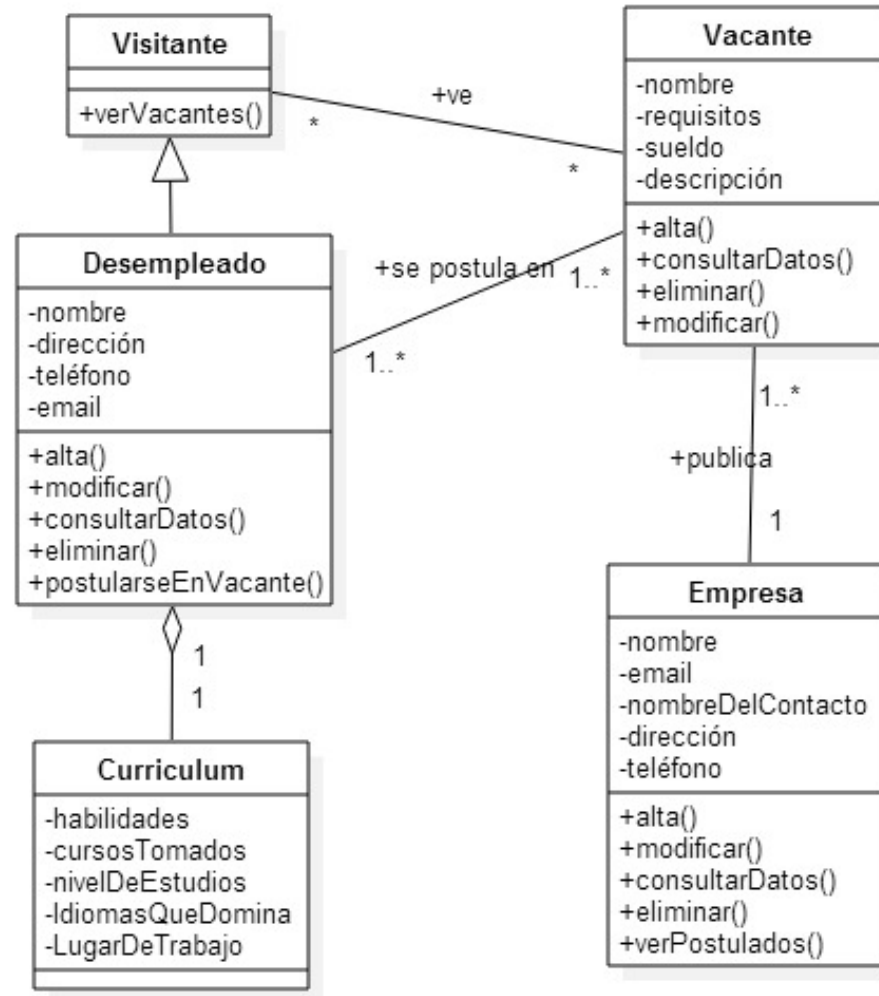
03. Diseño de componentes

- Proceso para identificar clases
 - Seleccionar un caso de uso y sus flujos de eventos
 - Por cada paso, se analizan los sustantivos más significativos como candidatos a clases. En la tarjeta se escribe el nombre de la clase. Ejemplo: alumno, libro
 - Cada acción asociada a la clase A es una responsabilidad de la clase A y se anota en la columna responsabilidad
 - La descripción de una responsabilidad deberá empezar con un verbo en modo infinitivo
 - Se analiza la responsabilidad R y se trata de identificar qué otras clases deben colaborar con la clase A para poder cumplir con esta responsabilidad
 - Si se identificó colaboración con alguna otra clase, anotarla en la columna colaboración
 - El proceso continúa para todos los pasos del caso de uso
 - El proceso termina cuando se hayan analizado todos los casos de uso

03. Diseño de componentes

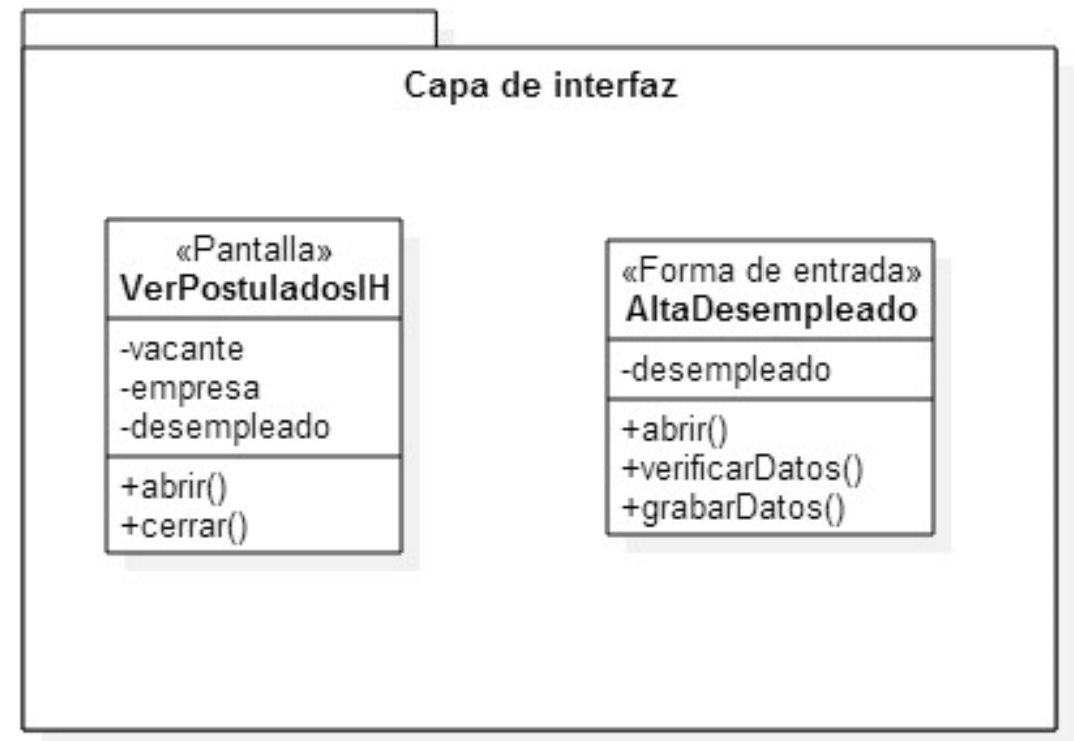
- Construir el diagrama de clases
 - Para cada tarjeta CRC se crea una clase
 - El mismo nombre que existe en la tarjeta, en singular y empezando con letra mayúscula
 - Por cada responsabilidad de la clase, se define el método correspondiente
 - Generalmente es un verbo en infinitivo e inicia con letra minúscula
 - Por cada colaboración entre clases se dibuja una asociación entre clases
 - Se especifica el nombre de la relación y la cardinalidad de los extremos de la asociación

Ejemplo de diagrama de clases



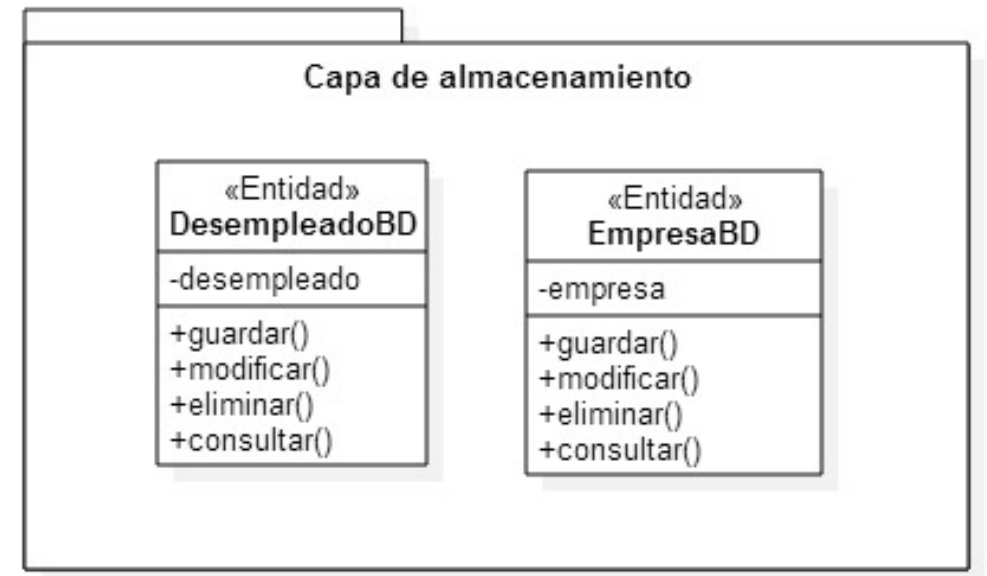
03. Diseño de componentes

- Identificación de clases de interfaz
 - Revisar el prototipo de interfaz de usuario creado en la fase de especificación de requisitos
 - Dibujar una clase por cada pantalla
 - Se puede utilizar el estereotipo <<Pantalla>> para las interfaces que muestran información
 - Se puede utilizar el estereotipo <<Forma de entrada>> cuando se trate de una clase de captura de datos



03. Diseño de componentes

- Identificar clases de entidad (o persistencia)
 - Se revisan las clases de control (de la capa de dominio del negocio) para identificar los atributos que deberán ser guardados.
 - Se crea una clase tipo entidad para aquellas clases que tengan atributos por guardar



03. Diseño de componentes

- Vista dinámica
 - Describe la interacción de los objetos para lograr la funcionalidad requerida del sistema
 - Existen distintos diagramas que modelan el comportamiento de los objetos
 - Diagramas de secuencia
 - Describen la interacción entre los objetos que intervienen en los escenarios de cada caso de uso
 - Diagramas de estado
 - Modelan el cambio de estados de las entidades del sistema

03. Diseño de componentes

- Diagramas de secuencia
 - Muestran la interacción entre objetos como una secuencia de envíos de mensajes entre ellos ordenados en el tiempo
 - Los elementos del diagrama son
 - Actor
 - Inicia la ejecución de la secuencia de mensajes que se envían entre los objetos
 - Objetos
 - Instancias de las clases. El objeto se representa como un rectángulo con el nombre del objeto: nombre de la clase. La línea vertical representa la línea de tiempo
 - Mensajes
 - Especificación de una comunicación entre objetos que transmite información con la expectativa de que se desencadenará alguna actividad (Booch, 2006). Se representa con una línea con flecha dirigida al objeto que recibe el mensaje. Sobre el nombre de la línea se escribe el mensaje
 - Los mensajes fluyen de izquierda a derecha y de arriba abajo.

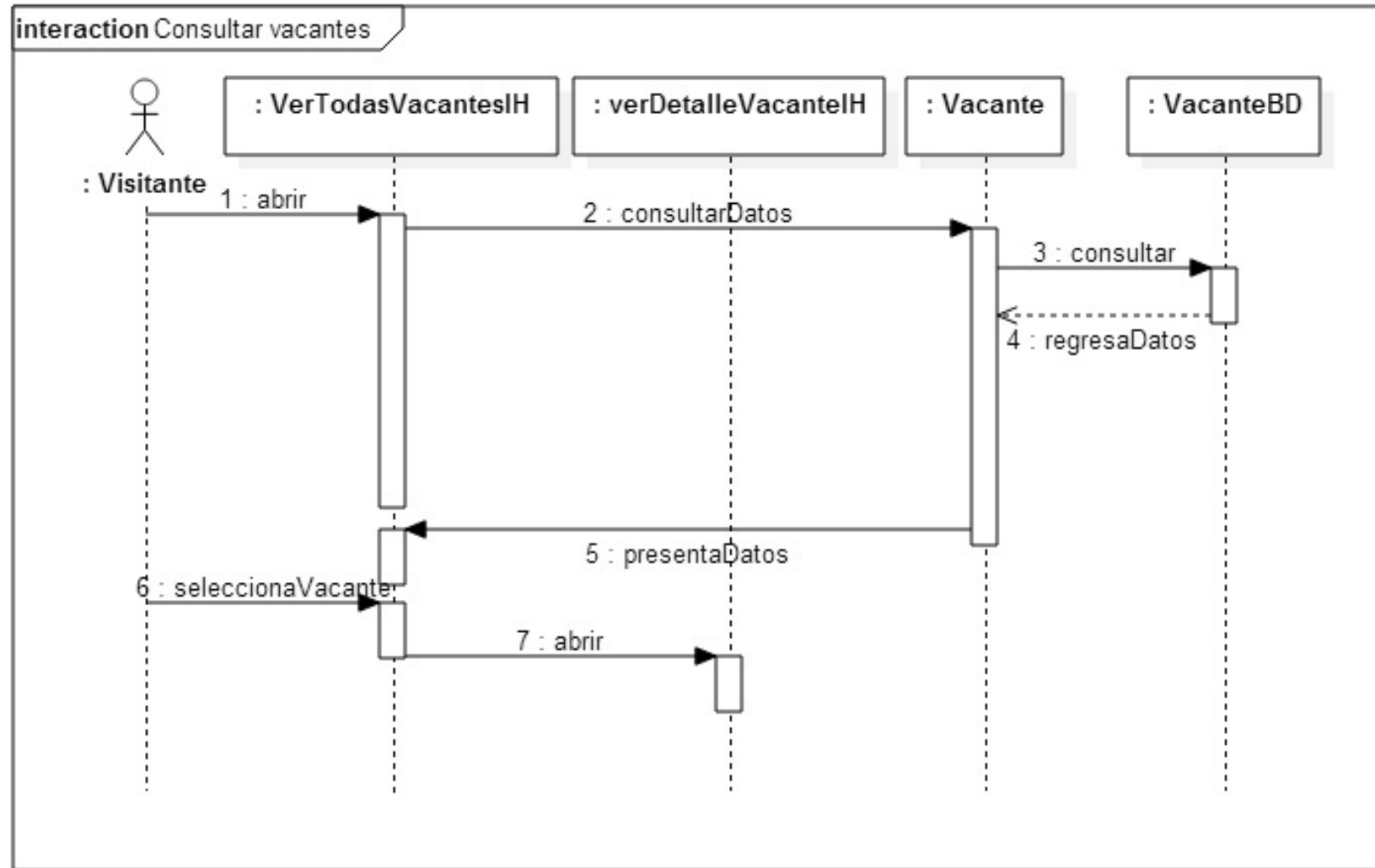
03. Diseño de componentes

- Diagrama de secuencia para los casos de uso
 - Para cada escenario del caso de uso se construye un diagrama de secuencia
 - Estos diagramas representan la vista dinámica de los casos de uso
 - Para cada escenario se identifican las clases participantes (interfaz, control, entidad) de las cuales se instanciarán los objetos necesarios
 - Se representa al actor que inicia el caso de uso
 - El actor envía un mensaje a un objeto de la clase interfaz identificada en el escenario
 - Siguiendo el flujo del escenario, se colocan los objetos identificados
 - Se envían los mensajes descritos en el flujo de eventos

03. Diseño de componentes

- Reglas para diseñar diagramas de secuencias a partir de los casos de uso
 - Los actores envían mensajes sólo a los objetos de las clases del tipo interfaz
 - Los objetos de las clases del tipo interfaz sólo pueden enviar mensajes a los objetos de la clase tipo control
 - Los objetos de la clases entidad sólo pueden enviar mensajes a los objetos de la clase tipo control
 - Los objetos de la clase tipo control pueden enviar mensajes a los objetos que son instancias de las clases de interfaz, de control y de entidad
 - Los objetos de la clase tipo control no envían mensajes a los actores

03. Diseño de componentes



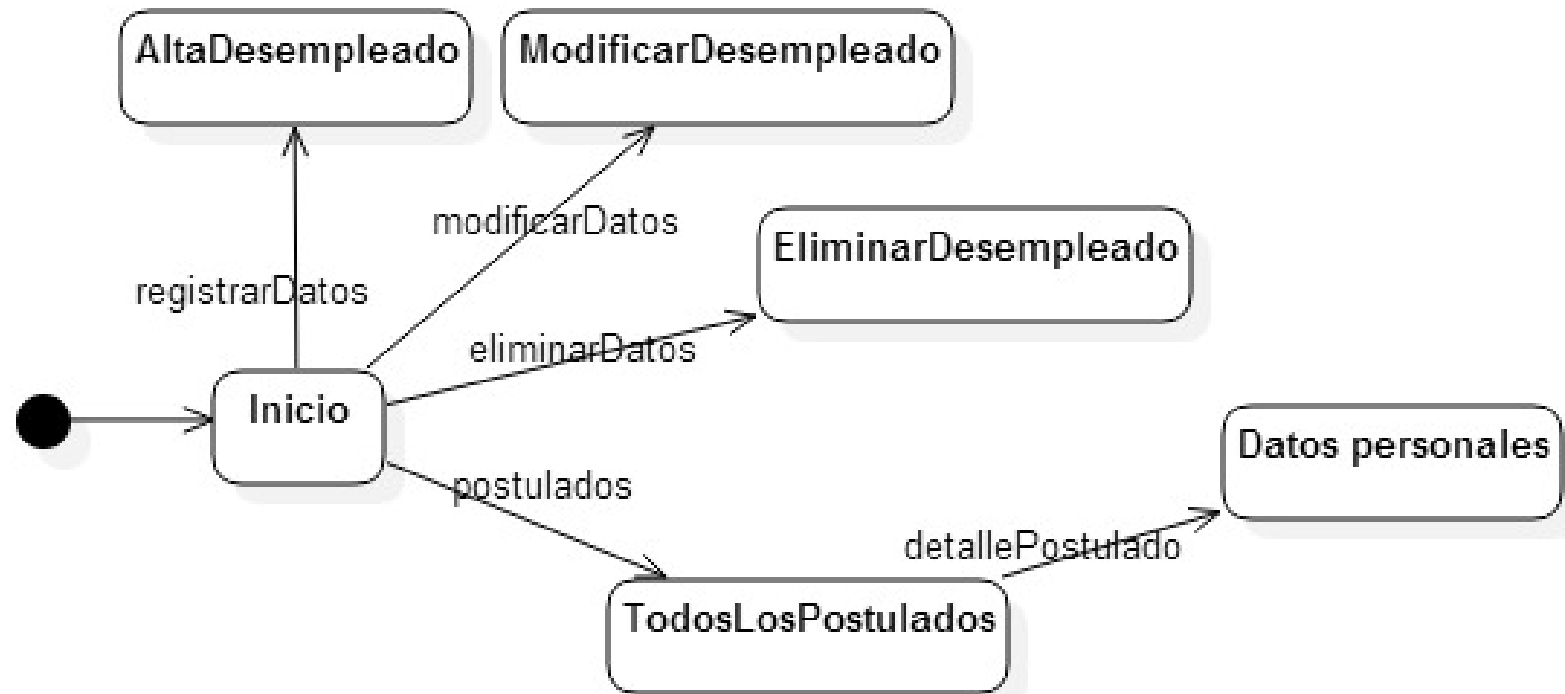
03. Diagrama de estados

- Diagrama de estados
 - Modela una máquina de estados finitos o autómata que enfatiza el flujo de control de un estado a otro.
 - Es una gráfica en donde los nodos representan estados y los arcos, transiciones
 - Elementos del diagrama
 - Estado
 - Condición de una entidad del sistema. Se representan por rectángulos con esquinas redondeadas. El nombre del estado se escribe dentro del rectángulo
 - Condición o situación en la vida de un objeto durante la cual satisface una condición, realiza alguna actividad o espera algún evento
 - Evento
 - Es la aparición de un estímulo que puede disparar una transición de estado
 - Transición
 - Relación entre dos estados que indica que un objeto en el primer estado realizará ciertas acciones y pasará al segundo estado cuando ocurran un evento específico y se satisfagan ciertas condiciones. Se representa con una flecha con el nombre del evento
 - Estado inicial (representado por un círculo de color negro)
 - Estado final (representado por dos círculos concéntricos, el del centro de color negro y el exterior sólo muestra la circunferencia)

03. Diseño de componentes

- Diagrama de estados para la navegación
 - Navegación en la interfaz de usuario
 - Es el cambio de pantallas que ve el usuario a causa de eventos como la selección de una opción de menú u oprimir un botón
 - Proceso para construir el diagrama
 - Las pantallas diseñadas en la especificación de requisitos representan estados
 - Los cambios entre las pantallas se darán por acciones que ejecute el usuario o sistema. Estas se modelarán como transiciones entre estados
 - El estado inicial del sistema apunta a la pantalla principal.
 - Esta pantalla generalmente tiene varias opciones de navegación
 - Cada opción de navegación representa un evento que podrá llevar a otra pantalla

03. Diseño de componentes



03. Diseño de componentes

- Diseño de base de datos
 - Se toman como base las clases de tipo entidad de la capa de almacenamiento para diseñar la base de datos
 - Se utilizan los conocimientos y técnicas de la asignatura de Base de Datos para implementar los mecanismos de persistencia del sistema
 - El diagrama Entidad-Relación se usa frecuentemente para diseñar la base de datos
 - Se analizan las clases relacionadas con la persistencia y se convierten en entidades
 - Los atributos de las clases se convierten en los atributos de las entidades
 - Las relaciones entre las entidades se derivan de las relaciones entre clases
 - Aplicar las reglas de normalización apropiadas

O4. Plan de pruebas de integración

- Plan de pruebas de integración
 - Describe el orden en que se juntan los elementos de los paquetes de la arquitectura
 - Se proponen las pruebas que se aplicarán para verificar la correcta interacción entre estos elementos
 - Este plan ayuda a revisar y completar el diseño arquitectónico
- Estrategias para la integración del sistema
 - Estrategia de paquetes
 - Se integran según los contenidos de cada paquete
 - Estrategia por funcionalidad o casos de uso
 - Se integran los elementos de las tres capas que correspondan a la realización de cada caso de uso
 - La integración empieza con los elementos que no dependan de otros

Referencias

- El material se basa principalmente en:
 - Ibargüengoitia, M. G., Oktaba, H. (2010). Ingeniería de Software Pragmática. UNAM.
 - Pressman, R. (2005). Ingeniería de software: un enfoque práctico. 6ta. Ed. McGraw-Hill. México.
 - Seidl, M., Scholz, M., Huemer, C., Kappel, G. (2015). UML @ Classroom: An Introduction to Object-Oriented Modeling. Springer Publishing Company, Incorporated.
 - Booch, G., Maksimchuk, R.A., Engle, M.W., Young, B.J., Conallen, J., Houston Kelli, A. (2007) Object- Oriented Analysis and Design with Applications (3rd Ed.). Addison Wesley Longman, Boston, MA, USA.